DSA </>

QUEUE

DATA STRUCTURE

FIFO | FIRST IN FIRST OUT

- EASY TO UNDERSTAND
- EXAMPLES INCLUDED
- C CODE INCLUDED
- INTERVIEW FRIENDLY

MASTER QUEUE IN DSA!

SUBSCRIBE NOW!

What is Queue?

- A **Queue** is a linear data structure in which insertion of elements is performed from one end called **REAR**, and deletion of elements is performed from another end called **FRONT**.
- Queue follows the **FIFO (First In First Out)** principle.
- This means:
  - The element inserted first will be removed first.
- **Real-Life Example of Queue**
  - People standing in a ticket counter line.
  - Printer queue.
  - CPU task scheduling.

Basic Operations of Queue

Operation	Description
Enqueue	Insert element into queue
Dequeue	Remove element from queue
Peek/Front	Display front element
Display	Show all queue elements

QUEUE OPERATIONS: INSERT (ENQUEUE) AND DELETE (DEQUEUE)

QUEUE (LINEAR)

- Follows FIFO (First In First Out)
- Insertion is allowed at REAR (Enqueue)
- Deletion is allowed at FRONT (Dequeue)

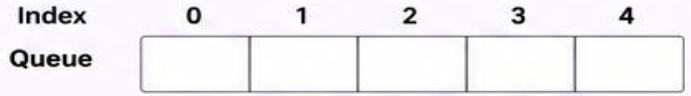
INITIAL STATE

Empty Queue

FRONT = −1, REAR = −1

QUEUE CAPACITY

Max Size = 5



FRONT = −1, REAR = −1

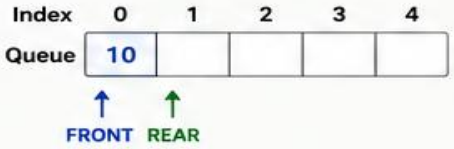
INSERT (ENQUEUE) OPERATIONS

Insert element at REAR and update REAR.

1 Enqueue(10)

Queue is Empty

Insert 10



FRONT = 0, REAR = 0

2 Enqueue(20)

Insert 20

FRONT = 0, REAR = 1

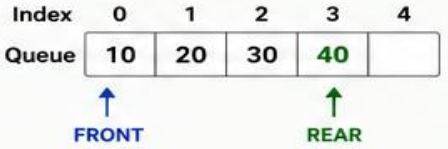
3 Enqueue(30)

Insert 30

FRONT = 0, REAR = 2

4 Enqueue(40)

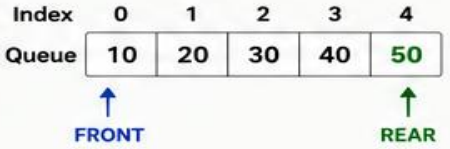
Insert 40



FRONT = 0, REAR = 3

5 Enqueue(50)

Insert 50



FRONT = 0, REAR = 4

Queue is FULL

DELETE (DEQUEUE) OPERATIONS

Delete element from FRONT and update FRONT.

1 Dequeue()

Delete 10

FRONT = 1, REAR = 4

2 Dequeue()

Delete 20

FRONT = 2, REAR = 4

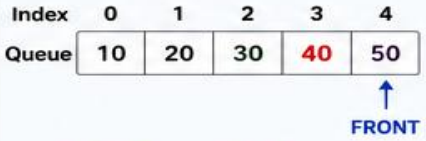
3 Dequeue()

Delete 30

FRONT = 3, REAR = 4

4 Dequeue()

Delete 40



FRONT = 4, REAR = 4

5 Dequeue()

Delete 50

Queue is Empty

FRONT = −1, REAR = −1

SUMMARY

- Enqueue: Add element at REAR.
- Dequeue: Remove element from FRONT.
- If REAR reaches Max Size − 1, Queue is FULL.
- If FRONT and REAR are −1, Queue is EMPTY.

STATE TABLE EXAMPLE

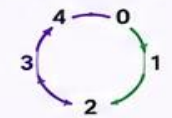
Operation	Element	FRONT	REAR	Queue (Size 5)
Initial	-	−1	−1	[_ _ _ _ _]
Enqueue	10	0	0	[10 _ _ _ _]
Enqueue	20	0	1	[10 20 _ _ _]
Enqueue	30	0	2	[10 20 30 _ _]
Dequeue	-	1	2	[_ 20 30 _ _]
Dequeue	-	2	2	[_ _ 30 _ _]
Dequeue	-	−1	−1	[_ _ _ _ _] (Empty)

IMPORTANT NOTES

- Queue follows FIFO (First In First Out).
- Insert at Rear → Enqueue
- Delete from Front → Dequeue
- Max Size = 5 (Index 0 to 4)

CIRCULAR QUEUE (NOTE)

In Circular Queue, when REAR reaches the last index, it wraps around to the beginning (0th index) if space is available.



Program

```
queue = []
# Enqueue Function
def enqueue():
    item = input("Enter element to enqueue: ")
    queue.append(item)
    print(item, "inserted into queue.")

# Dequeue Function
def dequeue():
    if len(queue) == 0:
        print("Queue Underflow! Queue is empty.")
    else:
        print("Deleted element:", queue.pop(0))

# Display Function
def display():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Queue elements:")
        for item in queue:
            print(item)
```

```
# Main Program
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        display()
    elif choice == 4:
        print("Program Ended.")
        break
    else:
        print("Invalid Choice!")
```

Types of Queue

- Simple Queue
- Circular Queue
- Priority Queue
- Double Ended Queue (Deque)

CIRCULAR QUEUE

Circular Queue is a linear data structure in which the last position is connected to the first position to form a circle. It follows **FIFO (First In First Out)** principle.

FEATURES

- The last position is connected to the first position.
- When **REAR** reaches the last position, the next insertion is done at the beginning if space is available.
- Efficient use of space.

OPERATIONS

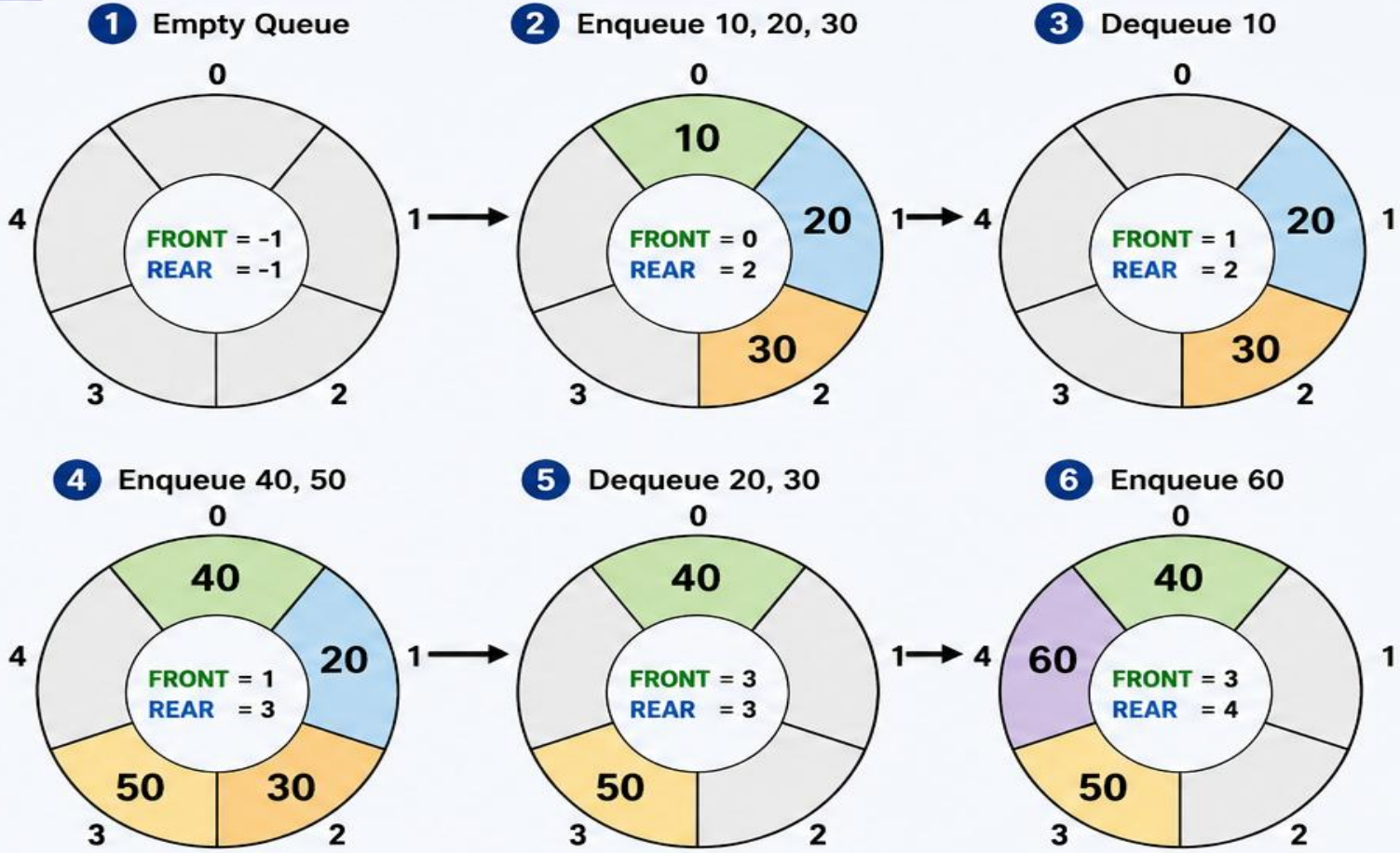
- **ENQUEUE** – Insert an element at REAR
- **DEQUEUE** – Delete an element from FRONT
- **FRONT** – Get the front element
- **REAR** – Get the rear element
- **ISEMPTY()** – Check if queue is empty
- **ISFULL()** – Check if queue is full

CONDITIONS

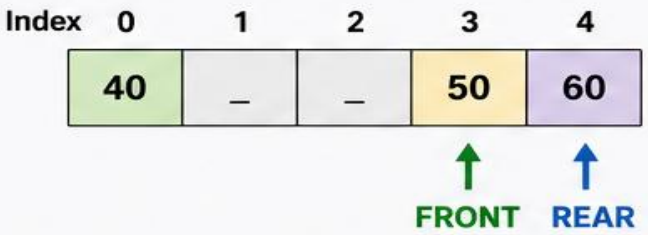
- Queue is Full : **(REAR + 1) % N == FRONT**
- Queue is Empty : **FRONT == -1**

(Where N is the size of the queue)

HOW CIRCULAR QUEUE WORKS



ARRAY REPRESENTATION



ADVANTAGES

- Overcomes the wastage of space in simple queue.
- Efficient utilization of array.
- All operations are performed in O(1) time.

N = Size of Queue = 5

Program

```
SIZE = 5
queue = [None] * SIZE
front = -1
rear = -1
# Enqueue Function
def enqueue():
    global front, rear
    if (rear + 1) % SIZE == front:
        print("Queue Overflow!")
        return
    item = input("Enter element: ")
    if front == -1:
        front = rear = 0
    else:
        rear = (rear + 1) % SIZE
    queue[rear] = item
    print(item, "inserted into queue.")
# Dequeue Function
def dequeue():
    global front, rear
    if front == -1:
        print("Queue Underflow!")
        return
    print("Deleted element:", queue[front])
    if front == rear:
        front = rear = -1
    else:
        front = (front + 1) % SIZE
```

```
# Display Function
def display():
    global front, rear
    if front == -1:
        print("Queue is empty.")
        return
    print("Queue elements:")
    i = front
    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i + 1) % SIZE
    print()
# Main Program
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        display()
    elif choice == 4:
        print("Program Ended.")
        break
    else:
        print("Invalid Choice!")
```

Applications of Queue

- CPU Scheduling
- Printer Management
- Call Center Systems
- BFS (Breadth First Search)
- Network Packet Handling